

Introdução ao SOLID e o Princípio da Responsabilidade Única (SRP)

1. O Conceito Geral de SOLID

Na engenharia de software, a entropia é uma força constante. À medida que os sistemas crescem e novos requisitos são adicionados, o código tende a tornar-se frágil, rígido e difícil de compreender. Para combater esta degradação arquitetural, o engenheiro de software Robert C. Martin (conhecido como "Uncle Bob") compilou e popularizou um conjunto de cinco princípios fundamentais de design orientado a objetos, consolidados sob o acrônimo **SOLID**.

O objetivo primordial do SOLID não é criar regras dogmáticas, mas sim fornecer métricas qualitativas e diretrizes para a criação de arquiteturas de software que sejam:

- **Tolerantes a mudanças:** Alterações em uma parte do sistema não quebram outras partes isoladas.
- **Compreensíveis:** O propósito de cada módulo é claro, reduzindo a carga cognitiva do desenvolvedor.
- **Testáveis e Reutilizáveis:** Componentes bem delimitados podem ser validados individualmente e reaproveitados em diferentes contextos da aplicação.

Os cinco princípios são:

- **S** - *Single Responsibility Principle* (Princípio da Responsabilidade Única)
- **O** - *Open/Closed Principle* (Princípio do Aberto/Fechado)
- **L** - *Liskov Substitution Principle* (Princípio da Substituição de Liskov)
- **I** - *Interface Segregation Principle* (Princípio da Segregação de Interfaces)
- **D** - *Dependency Inversion Principle* (Princípio da Inversão de Dependência)

Nesta aula, o nosso foco será o alicerce de todo o design de software: o Princípio da Responsabilidade Única.

2. O Princípio da Responsabilidade Única (SRP)

O Princípio da Responsabilidade Única (SRP) estabelece uma regra categórica: **"Uma classe deve ter um, e apenas um, motivo para mudar."**

Uma interpretação ingênua deste princípio sugere que uma classe deve fazer apenas "uma coisa". No entanto, a definição arquitetural correta está ligada à **coesão** e aos **atores** que

solicitam as mudanças. Se uma classe possui métodos que atendem às necessidades de diferentes departamentos de uma empresa (por exemplo, o setor financeiro e o setor de infraestrutura de TI), ela possui múltiplos motivos para mudar e, portanto, viola o SRP.

Quando uma classe acumula responsabilidades:

1. Ocorre o acoplamento indesejado entre lógicas distintas.
2. Aumenta a probabilidade de conflitos de integração (*merge conflicts*) em equipes grandes.
3. A alteração de um detalhe de infraestrutura (como o banco de dados) pode introduzir bugs críticos em regras de negócio puras.

3. Exemplos Práticos: O Antipadrão vs. A Solução Arquitetural

Abaixo, apresento três cenários comuns na construção de software corporativo em Java, evidenciando a violação estrutural e a respectiva refatoração orientada ao SRP.

Exemplo 1: Mistura de Domínio e Persistência (Active Record vs. Data Mapper)

O que está errado (Violação do SRP):

A classe `Empregado` abaixo acumula duas responsabilidades (dois motivos para mudar): as regras de negócio inerentes a um empregado (cálculo de bonificação) e a responsabilidade de infraestrutura (saber como inserir dados em um banco de dados relacional). Se a regra de bonificação mudar (solicitado pelo RH) ou se o banco de dados migrar para NoSQL (solicitado pela TI), esta mesma classe precisará ser alterada.

```
// ANTIPADRÃO - Alta violação de coesão
public class Empregado {
    private String nome;
    private double salarioBase;

    public Empregado(String nome, double salarioBase) {
        this.nome = nome;
        this.salarioBase = salarioBase;
    }

    // Motivo para mudar 1: Regra de Negócio
    public double calcularBonificacao() {
        return this.salarioBase * 1.15;
    }
}
```

```
// Motivo para mudar 2: Persistência de Dados
public void salvarNoBanco() {
    String sql = "INSERT INTO empregados (nome, salario) VALUES (" + this.nome + ", " +
this.salarioBase + ")";
    System.out.println("Executando: " + sql);
}
}
```

O que está certo (Aplicação do SRP):

Separamos o conceito nas camadas apropriadas. A classe de Domínio foca apenas no estado e comportamento do negócio. A persistência é extraída para um Repositório dedicado.

```
// SOLUÇÃO - Classe de Domínio Pura
public class Empregado {
    private String nome;
    private double salarioBase;

    public Empregado(String nome, double salarioBase) {
        this.nome = nome;
        this.salarioBase = salarioBase;
    }

    public double calcularBonificacao() {
        return this.salarioBase * 1.15;
    }

    // Getters omitidos
}
```

```
// SOLUÇÃO - Especialista em Persistência
public class EmpregadoRepository {
    public void salvar(Empregado empregado) {
        // Lógica exclusiva de acesso a dados (JDBC, JPA, etc.)
        System.out.println("Salvando o empregado no banco de dados isoladamente.");
    }
}
```

Exemplo 2: Extração de Dados vs. Formatação de Apresentação

O que está errado (Violação do SRP):

A classe `RelatorioFinanceiro` consolida os dados das vendas, mas também decide como esses dados serão formatados visualmente. Se a diretoria pedir uma nova fórmula de cálculo de lucros, ou se a equipe de marketing pedir que o relatório seja em HTML em vez de texto puro, a mesma classe será impactada.

```
// ANTIPADRÃO - Mistura lógica de processamento com lógica de apresentação
import java.util.List;

public class RelatorioFinanceiro {
    private List<Double> vendas;

    public RelatorioFinanceiro(List<Double> vendas) {
        this.vendas = vendas;
    }

    // Motivo para mudar 1: Lógica de consolidação de dados
    public double calcularReceitaTotal() {
        double total = 0;
        for (Double venda : vendas) {
            total += venda;
        }
        return total;
    }

    // Motivo para mudar 2: Lógica de apresentação (UI/Output)
    public String gerarRelatorioHTML() {
        double total = calcularReceitaTotal();
        return "<html><body><h1>Relatório: </h1><p>Total: $" + total + "</p></body></html>";
    }
}
```

O que está certo (Aplicação do SRP):

Delegamos a responsabilidade de formatação para uma entidade específica (um serializador ou formatador). O `RelatorioFinanceiro` apenas processa a informação.

```
// SOLUÇÃO - Foco apenas nos dados e cálculos
public class RelatorioFinanceiro {
```

```

private List<Double> vendas;

public RelatorioFinanceiro(List<Double> vendas) {
    this.vendas = vendas;
}

public double calcularReceitaTotal() {
    double total = 0;
    for (Double venda : vendas) {
        total += venda;
    }
    return total;
}

// SOLUÇÃO - Foco apenas na apresentação
public class FormatadorRelatorioHTML {
    public String formatar(double receitaTotal) {
        return "<html><body><h1>Relatório: </h1><p>Total: $" + receitaTotal + "</p></body></html>";
    }
}

```

Exemplo 3: Orquestração de Fluxo vs. Responsabilidades Transversais (Cross-cutting)

O que está errado (Violação do SRP):

Em arquiteturas de serviços, é comum encontrar classes "Deus" (*God Classes*). A classe `ServicoRegistroUsuario` abaixo valida regras de string, aplica criptografia e envia comunicações de rede. O acoplamento é extremo e a testabilidade é praticamente nula.

```

// ANTIPADRÃO - Classe utilitária generalista (God Class)
public class ServicoRegistroUsuario {

    public void registrarUsuario(String email, String senha) {
        // Motivo para mudar 1: Regra de validação de input
        if (!email.contains("@") || !email.contains(".")) {
            throw new IllegalArgumentException("Email inválido");
        }

        // Motivo para mudar 2: Algoritmo de segurança/criptografia
    }
}

```

```
String senhaCriptografada = "hash_" + senha;
```

```
// Motivo para mudar 3: Comunicação externa
```

```
System.out.println("Salvando usuário: " + email + " com senha " + senhaCriptografada);
```

```
System.out.println("Enviando e-mail de boas-vindas via SMTP para: " + email);
```

```
    }  
}
```

O que está certo (Aplicação do SRP):

A classe de serviço deve atuar apenas como uma **orquestradora**. Ela delega as responsabilidades especializadas (validação, segurança, notificação) para componentes altamente coesos.

```
// SOLUÇÃO - Especialistas isolados
```

```
public class ValidadorEmail {  
    public boolean isValido(String email) {  
        return email.contains("@") && email.contains(".");  
    }  
}
```

```
public class CodificadorSenha {  
    public String codificar(String senhaOriginal) {  
        return "hash_" + senhaOriginal; // Ex: BCrypt, Argon2  
    }  
}
```

```
public class ServicoNotificacao {  
    public void enviarBoasVindas(String email) {  
        System.out.println("Enviando e-mail de boas-vindas para: " + email);  
    }  
}
```

```
// SOLUÇÃO - A classe orquestradora
```

```
public class ServicoRegistroUsuario {  
    private ValidadorEmail validador;  
    private CodificadorSenha codificador;  
    private ServicoNotificacao notificacao;
```

```
// Dependências injetadas
```

```
public ServicoRegistroUsuario(ValidadorEmail validador, CodificadorSenha codificador,
```

```
ServicoNotificacao notificacao) {  
    this.validador = validador;  
    this.codificador = codificador;  
    this.notificacao = notificacao;  
}  
  
public void registrarUsuario(String email, String senha) {  
    if (!validador.isValido(email)) {  
        throw new IllegalArgumentException("Email inválido");  
    }  
  
    String senhaProtegida = codificador.codificar(senha);  
  
    // Simulação de persistência usando o estado processado  
    System.out.println("Salvando usuário: " + email + " com senha " + senhaProtegida);  
  
    notificacao.enviarBoasVindas(email);  
}  
}
```